4.1–4.2 · Optimization II: constraints and multipliers, and the KOMO formulation of inverse kinematics and trajectory optimization

Sessions 4.1–4.2 add joint limits, obstacle clearance and exact goals to the optimization methods of sessions 3.1–3.2. Session 4.1 derives the conditions a constrained solution must satisfy. Session 4.2 develops methods for finding such solutions and applies them to a whole trajectory. The trajectory formulation uses functions of nearby time steps. It is called k-order Markov Optimization (KOMO).

You should know the two-link arm's forward-kinematics map φ and Jacobian $\mathbf{J}$ from sessions 2.3–2.4. You should also know gradient descent, Armijo backtracking, Newton's method and the damped Gauss-Newton step from sessions 3.1–3.2. Those sessions distinguished a solved task from a stationary point, a stalled step and an iteration limit. We will extend those stopping tests to constrained problems.

Vectors and matrices are bold. Individual constraint components g_i and h_j and individual multipliers λ_i and κ_j are scalars. Their stacks $\mathbf{g}$, $\mathbf{h}$, $\boldsymbol{\lambda}$ and $\boldsymbol{\kappa}$ are vectors. The scalar d counts decision variables, and n counts joints. Iterations use i; k denotes the number of earlier knots a trajectory feature depends on.

0. What you will be able to do afterwards

- You will write joint limits, obstacle clearance, torque bounds and exact goals as inequality or equality constraints. You will state the four Karush-Kuhn-Tucker (KKT) conditions and explain how they describe a constrained stationary point.
- You will identify active constraints in a small problem and calculate their multipliers from stationarity. You will use a multiplier to predict the local change in optimal cost when a constraint is relaxed.
- You will write inverse kinematics (IK) and a whole trajectory as nonlinear programs. You will explain why k-order features produce a sparse Jacobian and a block-banded normal matrix, which makes the Gauss-Newton linear solve cheaper.
- You will check feasibility, Lagrangian stationarity and complementarity separately. You will explain why a small objective gradient or a small accepted step cannot replace these tests.
- You will compare squared penalties, log barriers and augmented Lagrangians. You will read the executed Drake program and the documented RAI mapping, state their time and weight conventions, and distinguish verified numerical results from unexecuted API calls.

1. Motivation and analogy

The two-link arm can reach the target in the unconstrained IK example. Now restrict its elbow to sixty degrees in either direction. Both exact IK solutions exceed that range. The task changes from reaching the target exactly to finding the closest allowed tip position.

At that constrained solution, the objective gradient can remain nonzero. Its negative points toward an improvement that the elbow limit prevents. The constraint gradient supplies the direction needed to balance it. The coefficient of that constraint gradient is the multiplier. A boundary can be compared with a fence that prevents movement into the forbidden region. This analogy explains the sign of an inequality multiplier, but the balance here is an optimization condition rather than a physical contact force.

A constraint is active when the solution lies on its boundary. It can be inactive when the unconstrained solution already respects it. An exact goal is different because the solution must satisfy the equality at every feasible point. Its multiplier can have either sign.

We first derive these conditions and calculate the elbow solution by hand. Then we turn constraint violations into terms in an inner objective, so the unconstrained methods of sessions 3.1–3.2 can solve it. A squared penalty increases the cost of violations. A log barrier permits only strictly feasible iterates. An augmented Lagrangian combines a penalty with multiplier estimates that are updated between inner solves.

Finally, we optimize all trajectory knots together. Obstacle avoidance makes this problem nonconvex, so different initial paths can lead to different constrained stationary paths. The examples will compare their costs and residuals without claiming a global optimum.

---

2. Prerequisite ramp

A constraint gradient tells us how a small decision-variable change affects its value. For the upper elbow limit,

$$g(\mathbf{q}) = q_2 - q_2^{\max}, \qquad \nabla g = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A feasible change decreases the elbow angle. In degrees for this sign check,

$$\Delta\mathbf{q} = \begin{bmatrix} 0 \\ -1^\circ \end{bmatrix}, \qquad \nabla g^\top \Delta\mathbf{q} = -1^\circ < 0.$$

The numerical solvers use radians. An obstacle constraint has a similar sign convention. Let $\mathbf{p}$ be a point on the arm, $\mathbf{c}$ the obstacle center and r_o its radius. Away from the obstacle center,

$$\begin{aligned} g(\mathbf{z}) &= r_o - \|\mathbf{p}(\mathbf{z}) - \mathbf{c}\|, \\ \hat{\mathbf{n}} &= \frac{\mathbf{p} - \mathbf{c}}{\|\mathbf{p} - \mathbf{c}\|}, \\ \nabla g &= -\left(\frac{\partial \mathbf{p}}{\partial \mathbf{z}}\right)^{\top} \hat{\mathbf{n}}, \\ \nabla g^{\top} &= -\hat{\mathbf{n}}^{\top} \frac{\partial \mathbf{p}}{\partial \mathbf{z}}. \end{aligned}$$

The gradient is a column vector. Its transpose is the constraint-Jacobian row used in §4.10. For the unit-disc constraint,

$$g(\mathbf{z}) = \|\mathbf{z}\| - 1, \qquad \nabla g = \frac{\mathbf{z}}{\|\mathbf{z}\|}.$$

This gradient points outward. In both examples, increasing the constraint value moves toward the forbidden side because feasibility requires a nonpositive value.

A vector lies in the span of several gradients when it is a weighted sum of them. With one nonzero constraint gradient, this means the vector is parallel to that gradient. This observation will give the stationarity equation.

At equally spaced trajectory knots, the first and second differences are

$$\mathbf{q}_t - \mathbf{q}_{t-1}, \qquad \mathbf{q}_{t-2} - 2\mathbf{q}_{t-1} + \mathbf{q}_t.$$

Dividing them by the time step and its square gives finite-difference approximations to instantaneous velocity and acceleration. The first difference also gives the interval's average velocity exactly. Each expression involves at most three consecutive knots. This locality produces matrices whose nonzero entries lie close to the diagonal. Such matrices are called banded. For a matrix of size N and bandwidth b, a banded factorization costs

$$O(Nb^2), \qquad \text{compared with } O(N^3) \text{ for a dense factorization.}$$

We also use a bracket that equals one when its condition is true and zero otherwise. For example,

$$[g > 0]g^2 = \begin{cases} g^2, & g > 0, \\ 0, & g \le 0. \end{cases}$$

It penalizes only violations. Finally, relaxing a constraint changes the best achievable cost. Write that optimal cost as

$$f^*(b) = \min_{\mathbf{z}} \{ f(\mathbf{z}) : g(\mathbf{z}) \leq b \}.$$

Its derivative with respect to b measures the local value of relaxing the constraint. Section 4.4 relates this derivative to the multiplier.

---

3. Formal setup

3.1 The nonlinear program (deck 04_05)

We choose a decision vector to minimize a scalar objective while satisfying inequalities and equalities. The general problem is

$$\begin{aligned} \min_{\mathbf{z} \in \mathbb{R}^d} \quad & f(\mathbf{z}) \\ \text{subject to} \quad & g_i(\mathbf{z}) \leq 0, \quad i = 1, \ldots, m, \\ & h_j(\mathbf{z}) = 0, \quad j = 1, \ldots, l. \end{aligned} \tag{1}$$

The scalar m counts inequalities, and l counts equalities. The objective and constraint functions are differentiable, and we can evaluate their values and gradients. The feasible set contains every decision vector satisfying all constraints. An inequality is active when its value is zero and inactive when its value is negative.

Robotics requirement	Constraint	Type
Joint j stays within its range.	$q_j - q_j^{\max} \leq 0$, $q_j^{\min} - q_j \leq 0$	Inequality
A point on the arm stays outside a sphere.	$r_o - \|\mathbf{p}(\mathbf{q}) - \mathbf{c}\| \leq 0$	Inequality
Joint torque stays within its bound.	$\tau_j - \tau_j^{\max} \leq 0$, $-\tau_j - \tau_j^{\max} \leq 0$	Inequality
The tip reaches the exact goal.	$\varphi(\mathbf{q}) - \mathbf{y}^* = \mathbf{0}$	Equality
The arm ends at rest.	$\mathbf{q}_T - \mathbf{q}_{T-1} = \mathbf{0}$	Equality

3.2 The KKT conditions and the Lagrangian

At a local minimum, the cost gradient can be balanced by constraint gradients. This requires a regularity assumption. Here we assume that the equality gradients and the active inequality gradients are linearly independent at $\mathbf{z}^*$. This assumption is a constraint qualification. Under it, multipliers exist that satisfy

$$\nabla f(\mathbf{z}^*) + \sum_i \lambda_i \nabla g_i(\mathbf{z}^*) + \sum_j \kappa_j \nabla h_j(\mathbf{z}^*) = \mathbf{0}. \tag{2}$$

Equation (2) is stationarity. The other three conditions are primal feasibility, dual feasibility and complementarity:

$$g_i(\mathbf{z}^*) \le 0, \qquad h_j(\mathbf{z}^*) = 0, \tag{3}$$

$$\lambda_i \ge 0, \tag{4}$$

$$\lambda_i g_i(\mathbf{z}^*) = 0. \tag{5}$$

These are the KKT conditions. They are necessary under the stated qualification, but they are not sufficient for a minimum. Example (m) has a saddle and a constrained maximum satisfying every condition.

We collect the cost and the multiplier terms in one scalar function, the Lagrangian:

$$L(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\kappa}) = f(\mathbf{z}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{z}) + \boldsymbol{\kappa}^\top \mathbf{h}(\mathbf{z}). \tag{6}$$

Stationarity means its gradient with respect to the decision vector is zero. It does not mean the Lagrangian has a minimum there. Positive-definite curvature in every direction would establish a strict local minimum of the Lagrangian. The constrained second-order sufficient condition requires positive curvature only in the relevant feasible directions, so it does not imply that stronger claim.

The elbow example also shows why no global minimum of the Lagrangian is promised. On its upper-limit branch,

$$L(\mathbf{q}, \lambda^*) = f(\mathbf{q}) + \lambda^*(q_2 - 60^\circ).$$

Angles in this expression are evaluated in radians. Shifting the optimum's elbow by one negative full turn gives the same arm pose and therefore the same objective:

$$\mathbf{q}^* = (-3.48^\circ, 60^\circ), \qquad \mathbf{q}' = (-3.48^\circ, -300^\circ).$$

The Lagrangian falls from approximately 0.0024 to -0.072. Every additional negative full turn lowers it by

$$2\pi\lambda^* \approx 0.075.$$

`constrained_ik.py` prints this comparison. The augmented Lagrangian of §4.9 can supply the missing normal curvature under its stated local assumptions and a sufficiently large penalty weight.

A problem is infeasible when no decision vector satisfies its constraints. A local solver's failure to find a feasible point does not prove this. In particular, `ret.feasible` reports what the RAI run found. The elbow example's infeasibility is proved by its

reachable radius in §3.3.

3.3 Inverse kinematics as a nonlinear program (Spring 2026 deck 03, motion planning, p.53 and p.60)

Sessions 3.1–3.2 minimized task error and penalized the length of each proposed step. Here the goal becomes a hard equality, and the cost selects a configuration near a fixed reference posture $\mathbf{q}_{\text{ref}}$. The matrix $\mathbf{D}$ weights posture differences. We use the weighted norm convention

$$\|\mathbf{a}\|_{\mathbf{D}}^2 = \mathbf{a}^\top \mathbf{D}\mathbf{a}.$$

The constrained IK problem is

$$\begin{aligned} \min_{\mathbf{q}} \quad & \tfrac{1}{2}\|\mathbf{q} - \mathbf{q}_{\text{ref}}\|_{\mathbf{D}}^2 \\ \text{subject to} \quad & \varphi(\mathbf{q}) - \mathbf{y}^* = \mathbf{0}, \\ & \mathbf{q} \le \mathbf{q}^{\max}, \qquad \mathbf{q}^{\min} \le \mathbf{q}, \\ & r_o - \|\mathbf{p}_p(\mathbf{q}) - \mathbf{c}\| \le 0. \end{aligned} \tag{7}$$

The index p selects a point on the arm. The posture is fixed throughout the solve. It is distinct from the changing iterate and from the first trajectory knot $\mathbf{q}_0$ in §3.4. The essentials' posture term uses $\mathbf{q}_0$ for this fixed posture, so translate it to $\mathbf{q}_{\text{ref}}$ when comparing these notes.

Replacing only the target equality by a squared penalty gives

$$\tfrac{1}{2}\|\varphi(\mathbf{q}) - \mathbf{y}^*\|_{\mathbf{K}}^2 + \tfrac{1}{2}\|\mathbf{q} - \mathbf{q}_{\text{ref}}\|_{\mathbf{D}}^2.$$

This is the essentials' posture-weighted formulation. For an isotropic task weight, its relation to the penalty convention in (10) is

$$\mathbf{K} = 2\mu\mathbf{I}.$$

The step penalty of sessions 3.1–3.2 is a different objective because its reference moves with the current iterate.

For the two-link arm with the elbow restricted to sixty degrees in either direction, the reachable radius ranges from 0.608 to 0.700 m. The target lies at radius 0.539 m:

$$\mathbf{y}^* = (0.5, 0.2)\ \text{m}.$$

Thus (7) is infeasible with that elbow range. Section 4.6 keeps this target soft and finds the closest feasible tip. The trajectory in example (i) uses obstacle clearance instead of the elbow range. These are different constraint sets on the same arm.

The Spring 2026 motion-planning source also includes gaze, fixed-foot and balance requirements. Each enters the same program as a cost, equality or inequality feature.

3.4 Trajectory optimization and the k-order Markov structure (Spring 2026 deck 03, motion planning, p.61; Toussaint 2017)

A trajectory replaces one configuration with a sequence of configurations. The first knot is fixed, and the remaining T knots are optimized:

$$\mathbf{q}_{0:T} = (\mathbf{q}_0, \dots, \mathbf{q}_T).$$

A function of a few consecutive knots is called a feature. A k-order feature uses k earlier knots as well as the current knot. Order zero uses one configuration, order one uses two, and order two uses three. Position and distance features have order zero. Velocity and acceleration differences have orders one and two. An orientation-error feature can also have order zero, using the rotation-vector representation from the essentials. The numerical examples here use positions.

The general KOMO problem is

$$\begin{aligned} \min_{\mathbf{q}_{1:T}} \quad & \sum_t \tfrac{1}{2} \|\mathbf{r}_t(\mathbf{q}_{t-k:t})\|^2 \\ \text{subject to} \quad & \mathbf{g}_t(\mathbf{q}_{t-k:t}) \le \mathbf{0}, \\ & \mathbf{h}_t(\mathbf{q}_{t-k:t}) = \mathbf{0}, \qquad \text{for each } t. \end{aligned} \tag{8}$$

Our acceleration cost, final goal, final rest condition and obstacle clearance are

$$\begin{aligned} \mathbf{r}_t &= \mathbf{q}_{t-2} - 2\mathbf{q}_{t-1} + \mathbf{q}_t, \quad t = 1, \dots, T, \quad \mathbf{q}_{-1} := \mathbf{q}_0, \\ \mathbf{h} &= \begin{bmatrix} \varphi(\mathbf{q}_T) - \mathbf{y}^* \\ \mathbf{q}_T - \mathbf{q}_{T-1} \end{bmatrix}, \\ g_{t,p} &= r_o - \|\mathbf{p}_p(\mathbf{q}_t) - \mathbf{c}\|. \end{aligned} \tag{9}$$

The repeated initial knot represents a start from rest. With one free knot,

$$T = 1 \quad \Longrightarrow \quad \mathbf{r}_1 = \mathbf{q}_1 - \mathbf{q}_0, \qquad f = \tfrac{1}{2}\|\mathbf{q}_1 - \mathbf{q}_0\|^2.$$

This is the posture cost of (7) with the reference equal to the first knot and the weight matrix equal to identity. The final rest equality must be dropped in this case because it would force the only free knot to equal the start. Joint limits enter (8) as order-zero inequalities; the trajectory example omits them.

The acceleration cost allows the steps to increase and decrease gradually. In example (i), guess B gives joint steps rising from 1.83° to 9.62° and then falling to zero. Replacing it by a velocity cost gives nineteen equal steps of 6.73°, followed by a zero step imposed by final rest. That motion starts at full speed and stops abruptly. Its tip path differs by up to 4 cm. Run `komo_two_link.py --k1` to compare the two costs.

The physical velocity and acceleration depend on a time step:

$$\dot{\mathbf{q}}_t \approx \frac{\mathbf{q}_t - \mathbf{q}_{t-1}}{\Delta t}, \qquad \ddot{\mathbf{q}}_t \approx \frac{\mathbf{q}_{t-2} - 2\mathbf{q}_{t-1} + \mathbf{q}_t}{\Delta t^2}.$$

Equation (9) uses raw differences with a unit knot step. Its cost has units of radians squared, rather than squared physical acceleration. For the same motion, increasing the number of knots approximately scales this raw cost as the inverse cube of the knot count. The script's `--sweep` gives

T	10	20	40
$T^3 f$	7.08	7.56	7.62

The multipliers have different scaling. The rest multiplier scales approximately as the inverse square of T because the rest residual itself shrinks as the inverse of T. The goal multiplier scales as the inverse cube only while no obstacle constraint is active. Active contact knots can change with the discretization. Multiplying the only objective by a constant leaves its minimizers unchanged, but relative scaling matters when another soft cost is present. RAI's `addControlObjective` also scales features by the square root of the time step (§6).

Clearance constraints hold at the knots only. Joint-linear interpolation between knots 12 and 13 of the twenty-knot solution sends the tip 2.07 mm into the disc. The penetration is 8.74 mm with ten knots and 0.43 mm with forty knots.

Stacking the features gives a residual vector with decision vector

$$\mathbf{z} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_T \end{bmatrix} \in \mathbb{R}^{2T}.$$

Each Jacobian row touches the current knot and at most k earlier knots. Consequently the normal matrix is block-banded. Section 4.10 shows how this structure reduces the solve cost.

3.5 Three constructions that turn a constraint into a cost (decks 04_06, 04_07)

To use the unconstrained solvers from sessions 3.1–3.2, we construct an inner objective that incorporates constraints. A squared penalty adds the squares of violations:

$$S(\mathbf{z}; \mu) = f(\mathbf{z}) + \mu \sum_i [g_i(\mathbf{z}) > 0] g_i(\mathbf{z})^2 + \mu \sum_j h_j(\mathbf{z})^2. \tag{10}$$

A log barrier becomes unbounded as an inequality approaches its boundary from inside:

$$B(\mathbf{z};\mu) = f(\mathbf{z}) - \mu \sum_i \log(-g_i(\mathbf{z})). \tag{11}$$

The augmented Lagrangian adds multiplier terms as well as squared violations:

$$\begin{aligned} A(\mathbf{z};\boldsymbol{\lambda},\boldsymbol{\kappa},\mu) = f(\mathbf{z}) \\ + \sum_i [g_i(\mathbf{z}) > 0 \vee \lambda_i > 0]\big(\lambda_i g_i(\mathbf{z}) + \mu g_i(\mathbf{z})^2\big) \\ + \sum_j \big(\kappa_j h_j(\mathbf{z}) + \mu h_j(\mathbf{z})^2\big). \end{aligned} \tag{12}$$

Each method alternates an inner minimization with an outer update. The penalty method increases μ, and the barrier method decreases it. The augmented Lagrangian in our code keeps μ fixed and updates multipliers after each inner solve:

$$\lambda_i \leftarrow \max\big(\lambda_i + 2\mu g_i(\mathbf{z}), 0\big), \qquad \kappa_j \leftarrow \kappa_j + 2\mu h_j(\mathbf{z}). \tag{13}$$

This is the variant described by Toussaint (2014). The RAI `NLP_Solver` documentation also specifies a factor-five penalty increase per outer iteration and a step tolerance of 0.01. Those defaults were read from the API reference and the KOMO tutorial trace on 2026-09-28. RAI has not been executed in this environment.

A step tolerance does not bound distance to an optimum or agreement between two solvers. To compare runs, match the model, features, weights, time convention and start. Then compare feasibility, stationarity, cost and path separately. Example (j) demonstrates why two solvers can produce different stationary paths from the same start.

Some texts place a one-half factor on the squared penalty. Their update then omits our factor of two. The conventions agree after rescaling the penalty parameter:

$$\mu g^2 \quad \leftrightarrow \quad \tfrac{\tilde{\mu}}{2} g^2, \qquad \tilde{\mu} = 2\mu.$$

---

4. Derivation

4.1 Force balance: stationarity for one active inequality

Let $\mathbf{z}^*$ be a local minimum subject to one inequality. Suppose the constraint is active and its gradient is nonzero:

$$g(\mathbf{z}^*) = 0, \qquad \nabla g(\mathbf{z}^*) \neq \mathbf{0}.$$

The gradient points into the forbidden side. A direction $\mathbf{d}$ points into or along the feasible side to first order when its inner product with this gradient is nonpositive. At a minimum, such a direction cannot reduce the objective to first order:

$$\nabla f(\mathbf{z}^*)^\top \mathbf{d} \geq 0 \quad \text{for every } \mathbf{d} \text{ with } \nabla g(\mathbf{z}^*)^\top \mathbf{d} \leq 0. \tag{14}$$

For strict inward directions this follows from a small feasible straight step. For a tangent direction, add a small inward component, apply the strict result, and take its limit. This handles the fact that a straight tangent step need not stay inside a curved boundary.

Separate the objective gradient into a part parallel to the constraint gradient and a perpendicular part:

$$\nabla f(\mathbf{z}^*) = a\nabla g(\mathbf{z}^*) + \mathbf{w}, \qquad \mathbf{w}^\top \nabla g(\mathbf{z}^*) = 0.$$

Choose the negative perpendicular part as the direction. Equation (14) gives

$$\mathbf{d} = -\mathbf{w} \quad \Longrightarrow \quad -\|\mathbf{w}\|^2 \geq 0 \quad \Longrightarrow \quad \mathbf{w} = \mathbf{0}.$$

Thus the gradients are parallel. Next choose the inward gradient direction:

$$\mathbf{d} = -\nabla g(\mathbf{z}^*) \quad \Longrightarrow \quad -a\|\nabla g(\mathbf{z}^*)\|^2 \geq 0 \quad \Longrightarrow \quad a \leq 0.$$

Define the nonnegative multiplier as the negative of a. Then

$$\nabla f(\mathbf{z}^*) + \lambda \nabla g(\mathbf{z}^*) = \mathbf{0}, \qquad \lambda \geq 0. \tag{15}$$

The negative cost gradient is balanced by a correction in the negative constraint-gradient direction. If the constraint is inactive, every sufficiently small direction remains feasible, so the objective gradient is zero and the multiplier can be zero. Together, the active and inactive cases give complementarity (5).

With several constraints, the negative cost gradient is a nonnegative combination of active inequality gradients plus an unrestricted combination of equality gradients. The one-constraint argument does not prove that statement. The general result uses Farkas' lemma and a constraint qualification, such as the independent gradients assumed in §3.2. The frozen deck 04_05 contains the fuller argument.

4.2 Complementarity as logic

For each inequality, (5) requires either a zero multiplier, a zero constraint value, or both. If the active set were known, we could discard inactive inequalities and replace active inequalities by equalities. Stationarity and the remaining constraints would form a system with the same number of equations and unknowns:

$$d + \#\{\text{active inequalities}\} + l.$$

There are potentially 2^m candidate active sets. The methods of §3.5 avoid enumerating them. The squared penalty includes violated inequalities. The augmented Lagrangian includes a row if its inequality is violated or its multiplier is positive. The

barrier keeps every inequality strictly feasible and assigns each an inferred positive multiplier while its parameter remains positive (§4.8).

4.3 Equality constraints: a rail holds on either side, and κ has no sign

An equality allows movement along its surface but forbids movement off it. Assume a scalar equality has a nonzero gradient at the solution. Its first-order feasible directions satisfy

$$h(\mathbf{z}^*) = 0, \qquad \nabla h(\mathbf{z}^*)^\top \mathbf{d} = 0.$$

Both a tangent direction and its negative are velocities of feasible curves through the solution. Applying the local-minimum argument to both requires

$$\nabla f(\mathbf{z}^*)^\top \mathbf{d} = 0 \quad \text{for all } \mathbf{d} \perp \nabla h(\mathbf{z}^*).$$

Therefore a real multiplier exists such that

$$\nabla f(\mathbf{z}^*) = -\kappa \nabla h(\mathbf{z}^*).$$

Its sign is unrestricted because the objective can favor movement to either side of the equality surface. There is no complementarity condition for an equality.

For a vector equality, each scalar row has its own multiplier. The Jacobian stacks the transposed row gradients, giving

$$\mathbf{h}(\mathbf{q}) = \varphi(\mathbf{q}) - \mathbf{y}^* = \mathbf{0}, \qquad \nabla f + \left(\frac{\partial \mathbf{h}}{\partial \mathbf{q}}\right)^\top \boldsymbol{\kappa} = \mathbf{0}.$$

The Panda example (k) recovers these multipliers by least squares. The KKT explorer's rail example shows the sign changing as its target crosses the line. For its unit-gradient equality and quadratic distance cost,

$$h(\mathbf{z}) = \frac{z_1 + z_2 - 1}{\sqrt{2}}, \qquad \kappa^* = h(\mathbf{z}_t), \qquad \mathbf{z}^* = \mathbf{z}_t - \kappa^* \nabla h.$$

The multiplier equals the target's signed distance from the line. Treating the same line as an inequality replaces the multiplier by its nonnegative part. These distance interpretations depend on this cost and a unit constraint gradient; the general interpretation is sensitivity of optimal cost.

4.4 The multiplier is a price

Relax one active inequality by replacing its zero bound with b. Assume a local solution branch varies smoothly with b and keeps the same active set near zero. Write its solution and cost as $\mathbf{z}^*(b)$ and $f^*(b)$. Differentiating the active constraint gives

$$g(\mathbf{z}^*(b)) = b \quad \Longrightarrow \quad \nabla g(\mathbf{z}^*)^\top \frac{d\mathbf{z}^*}{db} = 1. \tag{16}$$

The chain rule and stationarity (15) then give

$$\frac{df^*}{db} = \nabla f(\mathbf{z}^*)^\top \frac{d\mathbf{z}^*}{db} = -\lambda \nabla g(\mathbf{z}^*)^\top \frac{d\mathbf{z}^*}{db} = -\lambda. \tag{17}$$

A small relaxation lowers the optimal cost at the multiplier's rate. For the elbow limit, the rate is 0.01192 square meters per radian, or 0.000208 square meters per degree. A whole radian is too large for this local prediction; it would predict a reduction larger than the entire cost. An inactive inequality has zero multiplier and no local cost change from relaxing it.

The smooth-branch assumption matters. In the two-sided elbow problem, both solution branches tie at zero relaxation. Relaxing the upper limit lowers the global optimum at the quoted rate. Tightening it switches the optimum to the mirror solution,

$$\mathbf{q} = (47.09^\circ, -60^\circ), \qquad f^* = 0.002433.$$

Thus the globally selected cost has a kink there. Each smooth branch separately still has the derivative (17). Example (d) checks the relaxation numerically, and the Panda example (k) checks another sensitivity.

For the disc, the optimal cost equals one-half the squared multiplier. Increasing its radius by 0.1 gives the first-order prediction

$$\Delta f^* \approx -0.1\lambda.$$

This is a local approximation while the same constraint remains active. If the target is less than 0.1 outside the disc, the relaxation makes it feasible and the entire cost disappears. The first-order prediction then exceeds the available reduction.

4.5 The disc problem in closed form (the module's first fence)

The disc example finds the feasible point closest to a supplied target:

$$\min_{\mathbf{z}} \ \tfrac{1}{2}\|\mathbf{z} - \mathbf{z}_t\|^2 \quad \text{subject to} \quad g(\mathbf{z}) = \|\mathbf{z}\| - 1 \le 0.$$

If the target is inside or on the disc, it is the solution and its multiplier is zero. Otherwise the solution lies on the unit circle. Stationarity gives

$$(\mathbf{z}^* - \mathbf{z}_t) + \lambda \frac{\mathbf{z}^*}{\|\mathbf{z}^*\|} = \mathbf{0}, \qquad \|\mathbf{z}^*\| = 1.$$

Substituting the unit norm yields

$$(1+\lambda)\mathbf{z}^* = \mathbf{z}_t \quad \Longrightarrow \quad \mathbf{z}^* = \frac{\mathbf{z}_t}{\|\mathbf{z}_t\|}, \qquad \lambda = \|\mathbf{z}_t\| - 1.$$

The inside and outside cases combine as

$$\mathbf{z}^* = \frac{\mathbf{z}_t}{\max(1, \|\mathbf{z}_t\|)}, \qquad \lambda^* = \max(0, \|\mathbf{z}_t\| - 1). \tag{18}$$

For this cost and unit-gradient constraint, the multiplier equals the target's distance outside the disc. The cost and its radius sensitivity are

$$f^* = \tfrac{1}{2}(\lambda^*)^2, \qquad df^* = -\lambda^*\, db.$$

4.6 The two-link arm with an elbow limit, in closed form

This example shows how an active joint limit prevents an exact reach. The link lengths are 0.4 and 0.3 m. We minimize the usual task-error cost subject to the elbow range:

$$\begin{aligned} f(\mathbf{q}) &= \tfrac{1}{2}\|\varphi(\mathbf{q}) - \mathbf{y}^*\|^2, \qquad \mathbf{y}^* = (0.5, 0.2)^\top \text{ m}, \\ g_1(\mathbf{q}) &= q_2 - 60^\circ \le 0, \\ g_2(\mathbf{q}) &= -60^\circ - q_2 \le 0. \end{aligned}$$

The formulas and solvers evaluate angles in radians. Without the limits, exact IK gives

$$\cos q_2 = \frac{\|\mathbf{y}^*\|^2 - l_1^2 - l_2^2}{2 l_1 l_2} = 0.1667, \qquad q_2 = \pm 80.41^\circ.$$

The positive-elbow solution violates the upper limit, and the negative-elbow solution violates the lower one. Consider the positive branch with the elbow fixed at sixty degrees. The two link vectors sum to

$$\varphi(\mathbf{q}) = \begin{bmatrix} l_1 \cos q_1 + l_2 \cos(q_1 + q_2) \\ l_1 \sin q_1 + l_2 \sin(q_1 + q_2) \end{bmatrix} = R \begin{bmatrix} \cos(q_1 + \delta) \\ \sin(q_1 + \delta) \end{bmatrix},$$

where the resultant length and angle are

$$R = \sqrt{l_1^2 + l_2^2 + 2 l_1 l_2 \cos q_2}, \qquad \delta = \operatorname{atan2}(l_2 \sin q_2, l_1 + l_2 \cos q_2).$$

With the elbow fixed, varying the first joint moves the tip around a circle of radius

$$R = \sqrt{0.37} = 0.6083 \text{ m} > \|\mathbf{y}^*\|.$$

The closest tip points in the target's direction, so

$$\begin{aligned} q_1^* &= \operatorname{atan2}(y_2^*, y_1^*) - \operatorname{atan2}(l_2 \sin q_2^*, l_1 + l_2 \cos q_2^*) \\ &= 21.80^\circ - 25.28^\circ = -3.48^\circ. \end{aligned} \tag{19}$$

The upper-limit gradient has only a second component. Stationarity therefore requires zero objective derivative in the free first joint, which (19) satisfies. The remaining equation determines the multiplier. Compute the task residual and second Jacobian column:

$$\mathbf{r} = \varphi(\mathbf{q}^*) - \mathbf{y}^* = 0.0698 \begin{bmatrix} \cos 21.80^\circ \\ \sin 21.80^\circ \end{bmatrix} \approx \begin{bmatrix} 0.0648 \\ 0.0259 \end{bmatrix} \ \mathrm{m},$$
$$\mathbf{J}_{:,2} = l_2 \begin{bmatrix} -\sin(q_1^* + q_2^*) \\ \cos(q_1^* + q_2^*) \end{bmatrix} \approx \begin{bmatrix} -0.2502 \\ 0.1655 \end{bmatrix} \ \mathrm{m/rad}.$$

Then

$$\begin{aligned} \lambda^* &= -\frac{\partial f}{\partial q_2} = -\mathbf{r}^\top \mathbf{J}_{:,2} \\ &\approx -\big(0.0648(-0.2502) + 0.0259(0.1655)\big) \\ &= 0.01192 \ \mathrm{m^2/rad}. \end{aligned} \tag{20}$$

The positive multiplier satisfies dual feasibility. Increasing the elbow bend would shorten the arm's reach toward the target, and the upper limit prevents it. The tip stops about 7 cm beyond the target.

The mirror branch has

$$q_2^* = -60^\circ, \qquad q_1^* = 21.80^\circ + 25.28^\circ = 47.09^\circ, \qquad \nabla g_2 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Its lower-limit multiplier is the positive derivative of the objective with respect to the elbow angle, again 0.01192. Both constrained minima have the same cost. Example (c) reports the mirror solution. The clamped iterate from sessions 3.1–3.2 lies on the same upper boundary but fails stationarity in the free joint:

$$\mathbf{q} = (-16.40^\circ, 60^\circ), \qquad \frac{\partial f}{\partial q_1} \approx -0.073.$$

Its cost exceeds four times the constrained minimum. Example (l) checks this failure.

4.7 The squared penalty: violation proportional to $1/\mu$

For one violated inequality, stationarity of the penalty inner problem at $\mathbf{z}'$ is

$$\nabla f(\mathbf{z}') + 2\mu g(\mathbf{z}') \nabla g(\mathbf{z}') = \mathbf{0}.$$

Comparing this with KKT stationarity identifies an approximate multiplier:

$$g(\mathbf{z}') \approx \frac{\lambda^*}{2\mu}. \tag{21}$$

A constraint with a positive limiting multiplier remains violated at finite penalty weight. Near the constrained solution, the violation shrinks approximately as the inverse of the weight. Example (f) gives elbow violations

$$0.34^\circ, \quad 0.034^\circ, \quad 0.0034^\circ, \quad 0.0003^\circ \quad \text{for } \mu = 1, 10, 100, 1000.$$

Increasing the weight also adds large curvature. The Gauss-Newton penalty term is

$$2\mu \nabla g \nabla g^\top.$$

When the other curvature scales remain bounded and nonzero, the condition number grows with the penalty weight. The code's Hessian model has condition numbers 5.9, 55, 541 and 5400 along example (f). The inner linear system becomes harder to solve accurately as the constraint approximation improves.

4.8 The log barrier: relaxed complementarity and the central path

At a stationary point of the barrier objective,

$$\nabla f(\mathbf{z}') - \sum_i \frac{\mu}{g_i(\mathbf{z}')} \nabla g_i(\mathbf{z}') = \mathbf{0}.$$

For positive barrier parameter and strictly feasible iterates, define

$$\lambda_i = -\frac{\mu}{g_i(\mathbf{z}')} > 0.$$

The inferred multipliers satisfy stationarity and dual feasibility. Primal feasibility holds strictly, but complementarity is relaxed:

$$\lambda_i g_i(\mathbf{z}') = -\mu. \tag{22}$$

Reducing the parameter makes this product approach zero. Under the usual regularity and local-minimum assumptions, the corresponding inner minima follow a solution branch toward a constrained optimum. This branch is called the central path. For convex problems with a strictly feasible point, the standard barrier theory provides the stronger global statement. A nonconvex stationary branch need not reach a minimum; §7 (6) gives a saddle counterexample.

The logarithm requires a strictly feasible start. It cannot represent an equality, which has no strict interior in the decision space. Interior-point methods keep equalities in their inner Newton system instead. Boyd and Vandenberghe's centering problem describes that extension; it is outside the main derivation here.

The barrier weight must be scaled to the objective. Its positive gradient-outer-product curvature term is

$$\sum_i \frac{\mu}{g_i^2}\nabla g_i \nabla g_i^\top = \sum_i \frac{\lambda_i^2}{\mu}\nabla g_i \nabla g_i^\top .$$

For a constraint with a positive limiting multiplier, this term grows without bound as the parameter approaches zero. For an inactive constraint with nonzero limiting slack, its multiplier shrinks with the parameter and the term vanishes. The code's Gauss-Newton Hessian model has condition numbers 14.6, 3.0, 4.5, 38.9 and 384 along example (g). It improves at first and then worsens. Toussaint (2014, §2) discusses this limiting conditioning problem.

The backup slide's objective-gap bound is

$$f(\mathbf{z}'(\mu)) - f^* \le m\mu.$$

It requires the convex barrier setting and an appropriately solved inner problem. At the nonconvex saddle of §7 (6), it fails by four orders of magnitude. It cannot certify that saddle as an optimum.

4.9 The augmented Lagrangian: learning the force

For one equality, the augmented objective and inner stationarity condition are

$$\begin{aligned} A &= f + \kappa h + \mu h^2, \\ \mathbf{0} &= \nabla f(\mathbf{z}') + \kappa \nabla h(\mathbf{z}') + 2\mu h(\mathbf{z}')\nabla h(\mathbf{z}') \\ &= \nabla f(\mathbf{z}') + \big(\kappa + 2\mu h(\mathbf{z}')\big)\nabla h(\mathbf{z}'). \end{aligned} \tag{23}$$

The coefficient in parentheses estimates the KKT multiplier. Update (13) stores that estimate for the next inner solve. The linear multiplier term then provides more of the required gradient balance, so less constraint violation is needed.

For affine cost and constraint functions, one multiplier update is exact and the next exact inner solve can satisfy the equality. In a nonlinear problem, local convergence at fixed penalty weight requires a sufficiently large weight and the usual regularity and second-order sufficient assumptions at the solution. Under those assumptions, violations decrease at a linear rate. Nocedal and Wright, chapter 17, gives the local result. The frozen deck 04_07 §3 works the quadratic one-dimensional example

$$f(z) = \tfrac{1}{2}z^2, \qquad h(z) = z - 1, \qquad \text{contraction factor } \frac{1}{1+2\mu}.$$

The algorithm alternates two operations. The inner solve holds multipliers fixed and changes the decision vector. Our code uses damped Gauss-Newton and restarts Armijo from a unit step. It reports and propagates a rejected direction as a stall. The outer update holds the decision vector fixed and applies (13).

For inequalities, the same balance applies while the row is included. The projected update prevents negative multipliers. A positive multiplier on a strictly satisfied inequality can occur during the iterations; at convergence, complementarity requires it to vanish if the inequality stays inactive. Fixed penalty weight avoids the progressively growing penalty curvature, but it does not remove the objective's own poor conditioning. RAI's documented defaults additionally increase the weight (§3.5).

To use Gauss-Newton, complete the square for an included inequality row:

$$\lambda g + \mu g^2 = \mu \left(g + \frac{\lambda}{2\mu} \right)^2 - \frac{\lambda^2}{4\mu}.$$

The last term is constant with respect to the decision vector during the inner solve. Every row with a positive multiplier is always included. Rows that can switch inclusion have zero multiplier and therefore zero square-completion constant. The sum of these constants is independent of the decision vector even when rows switch. In the half-squared-norm convention, the constraint residual and its Jacobian row are

$$r_g = \sqrt{2\mu} \left(g + \frac{\lambda}{2\mu} \right), \qquad \mathbf{J}_{r_g} = \sqrt{2\mu} \nabla g^\top.$$

An equality uses the same construction with its own multiplier:

$$r_h = \sqrt{2\mu} \left(h + \frac{\kappa}{2\mu} \right), \qquad \mathbf{J}_{r_h} = \sqrt{2\mu} \nabla h^\top.$$

Stack these with the objective residuals in $\mathbf{r}$. The inner objective is a half squared norm plus a constant. Away from a row switch, its residual Jacobian gives the damped step

$$\boldsymbol{\delta} = -(\mathbf{J}_r^\top \mathbf{J}_r + \rho \mathbf{I})^{-1} \mathbf{J}_r^\top \mathbf{r}, \qquad \mathbf{J}_r = \frac{\partial \mathbf{r}}{\partial \mathbf{z}}.$$

The scalar ρ is damping. Included inequality rows have positive violation or positive multiplier. At the first outer update in example (h), the stored residual shift equals that iteration's elbow violation:

$$\frac{\lambda}{2\mu} = 0.338^\circ.$$

The shifted residual explains how the same Gauss-Newton machinery handles the augmented objective.

4.10 Why the k-order structure makes the step cheap

With T free knots and n joints, the decision-vector dimension is

$$d = nT.$$

Acceleration rows touch three knots. Obstacle and goal-position rows touch one, and final-rest rows touch two. Every row touches the current knot and at most k earlier knots. The grouped rectangular Jacobian is a stack of sparse row blocks; it need not form one diagonal band. Its normal matrix is block-banded with scalar bandwidth of order

$$(k+1)n.$$

A banded factorization of the normal equations costs

$$O\big(T(k+1)^2 n^3\big), \qquad \text{rather than } O\big((nT)^3\big).$$

For a hundred knots, seven joints and order two, the corresponding operation counts are approximately

$$3 \times 10^5 \quad \text{and} \quad 3 \times 10^8,$$

a factor of about 1100. A dense matrix of size seven hundred can still be solved in about 2 ms on a laptop. The structural saving matters as the trajectory grows and when a solver repeats the step many times.

We distinguish two Jacobians. The full feature Jacobian contains every unscaled feature row. At twenty free knots and two joints, it has size

$$104 \times 40 = (40 \text{ smoothness} + 4 \text{ equality} + 60 \text{ inequality rows}) \times 40.$$

It has 5.3 percent nonzero entries and is drawn in `fig/komo_band.png`. The inner residual Jacobian scales equality and included inequality rows by the square root of twice the penalty weight. It includes only violated inequalities or inequalities with positive multipliers. It has size 46 by 40 at solution A, which includes two obstacle rows, and 44 by 40 at solution B, which includes none.

At a fixed configuration, penalty weight and included-row set, multiplier shifts change residual values without changing their Jacobian rows. Only the right-hand side of the normal equations changes. A projected update can remove an inequality row, so it can also change the normal matrix. Temporal locality survives either case. The educational code uses a dense solve because its normal matrix is only forty by forty; KOMO can exploit the banded structure.

The four feature block types in solution A are

$$\begin{aligned}
\text{acceleration:} &\quad [\mathbf{I},\ -2\mathbf{I},\ \mathbf{I}],\\
\text{final rest:} &\quad [-\mathbf{I},\ \mathbf{I}],\\
\text{goal position:} &\quad \mathbf{J}(\mathbf{q}_T) \approx \begin{bmatrix} -0.200 & -0.280 \\ 0.500 & 0.108 \end{bmatrix},\\
\text{obstacle:} &\quad -\,\hat{\mathbf{n}}^{\top}\mathbf{J}_p(\mathbf{q}_t).
\end{aligned}$$

Here each identity block is two by two. The acceleration Jacobian is constant, so Gauss-Newton is exact for that quadratic cost. The goal block's first column is the perpendicular target-position vector from the kinematic column rule:

$$\mathbf{J}_{:,1}(\mathbf{q}_T) = (-y_2^*, y_1^*)^\top.$$

For an obstacle row, $\hat{\mathbf{n}}$ points from the disc center to the sampled point and $\mathbf{J}_p$ is that point's position Jacobian. The elbow point does not depend on the second joint, so its second Jacobian column is zero. This is the missing entry in each three-row obstacle group of the figure. The inner solver scales constraint rows and shifts their residuals as in §4.9.

At solution A, the stacked residual's normal matrix has condition number

$$\operatorname{cond}(\mathbf{J}_r^\top \mathbf{J}_r) = 5.1 \times 10^{3}.$$

The smoothness rows alone have condition number

$$2.5 \times 10^{5}.$$

Their condition number grows approximately as the fourth power of the knot count, reaching

$$8.7 \times 10^{6} \text{ at } T = 50, \qquad 1.3 \times 10^{8} \text{ at } T = 100.$$

At the twenty-knot smoothness condition number, the quadratic gradient-descent model from session 3.1 contracts error by only 0.999992 per step. This is another reason to use the curvature and sparse structure.

4.11 Stopping with fences: three residuals, not one gradient

A constrained optimum can have a nonzero objective gradient. At the elbow solution its norm is 0.0119, and along trajectory A it is 0.027. Constraint gradients balance it. Conversely, a zero objective gradient does not establish feasibility or a minimum. We need separate KKT residuals.

Both scripts check the following quantities after each multiplier update:

$$\begin{aligned} \text{feasibility:} \quad & \max\left(\max_i g_i^+,\ \max_j |h_j|\right), \qquad g_i^+ = \max(g_i, 0), \\ \text{stationarity:} \quad & \left\| \nabla f + \sum_i \lambda_i \nabla g_i + \sum_j \kappa_j \nabla h_j \right\|, \\ \text{complementarity:} \quad & \max_i |\lambda_i g_i|. \end{aligned}$$

Their tolerances, in the same order, are

$$10^{-9}, \qquad 10^{-8}, \qquad 10^{-11}.$$

The projected multiplier update also enforces dual feasibility. The feasibility maximum combines the numerical units of its component rows. These tolerances are choices for the stated feature scaling, rather than universal physical tolerances.

An accurately minimized inner objective can make the updated Lagrangian gradient small, but the projection introduces an important qualification. Let $\mathcal{I}$ contain the included inequality rows. At the same configuration, away from a row switch and with no included multiplier clamped by the update,

$$\begin{aligned}\nabla_{\mathbf{z}} A &= \nabla f + \sum_{i \in \mathcal{I}} (\lambda_i + 2\mu g_i)\nabla g_i + \sum_j (\kappa_j + 2\mu h_j)\nabla h_j \\ &= \nabla_{\mathbf{z}} L(\mathbf{z}, \boldsymbol{\lambda}_{\text{new}}, \boldsymbol{\kappa}_{\text{new}}).\end{aligned}$$

An inner gradient tolerance then bounds the stationarity residual. A small accepted step alone does not. Clamping an included multiplier can break the identity. Excluded rows remain zero and contribute neither term.

The counterexample in `check_solvers.py` uses

$$f(z) = \tfrac{1}{2}(z+2)^2, \qquad g(z) = z \le 0, \qquad \lambda = 1, \qquad \mu = 1.$$

The included-row objective is

$$A(z) = \tfrac{1}{2}(z+2)^2 + z + z^2.$$

Its minimum is at -1, but the update clamps the multiplier:

$$z' = -1, \qquad \lambda_{\text{new}} = \max(1-2, 0) = 0, \qquad |\nabla f(z')| = 1.$$

The inner stationary point therefore has a nonzero updated stationarity residual.

The two inner solvers use different tests:

$$\begin{aligned}\text{elbow gradient tolerance} &= 10^{-9}, \\ \text{trajectory accepted-step tolerance} &= 10^{-10}.\end{aligned}$$

Neither inner test replaces the outer residual checks. The elbow run stops at outer iteration 4 with

$$6.5 \times 10^{-11}, \qquad 5.0 \times 10^{-11}, \qquad 7.7 \times 10^{-13}.$$

At iteration 3 those residuals are still

$$4.9 \times 10^{-9}, \qquad 1.3 \times 10^{-11}, \qquad 5.9 \times 10^{-11}.$$

Trajectory A stops after six outer iterations with

$$\max_j |h_j| = 5.1 \times 10^{-10}, \qquad \max_i g_i^+ = 0,$$

$$\text{stationarity} = 1.3 \times 10^{-10}, \qquad \text{complementarity} = 9.5 \times 10^{-13}.$$

The four outcomes from sessions 3.1–3.2 now apply to the constrained outer problem. A run is solved when all three residuals satisfy their tolerances. A stationary inner point may still have unsettled feasibility or complementarity. A stalled inner solve rejects its trial and passes the `inner stalled` outcome to the outer loop. A limit outcome means the outer iteration cap was reached.

These outcomes describe the tested conditions. Even a solved KKT system need not be a minimum, and failure to find a feasible point need not prove infeasibility. `check_solvers.py` checks that a rejected trial never replaces the current iterate and that stalls propagate through both outer solvers.

---

5. Worked examples

The supplied scripts print these values before rounding: `code/constrained_ik.py` prints examples (a)–(h), (l) and (m); `code/komo_two_link.py` prints (i)–(j); `drake/03-ik-as-nlp/ik_nlp_panda.py` prints (k). Example (a′) is also printed by `constrained_ik.py`. Section 6 gives the run commands.

(a) The disc problem (18). These targets show active and inactive cases. The first two targets lie outside the disc and project onto the circle:

$\mathbf{z}_t$	$\|\mathbf{z}_t\|$	$\mathbf{z}^*$	λ^*
$(2, 0)$	2	$(1, 0)$	1
$(1.2, 0.9)$	1.5	$(0.8, 0.6)$	0.5

The third target is inside the disc:

$$\mathbf{z}_t = (0.5, 0.3), \qquad \mathbf{z}^* = \mathbf{z}_t, \qquad \lambda^* = 0, \qquad g(\mathbf{z}^*) \approx -0.417.$$

All three have zero stationarity residual to machine precision and zero complementarity product:

$$\|\nabla f + \lambda \nabla g\| = 0, \qquad \lambda g = 0.$$

The problem and gradient-balance arrows are in `fig/kkt_disc.png` and `fig/kkt_disc_forces.png`. The KKT explorer lets you drag the target through these cases.

(a′) The rail, in closed form. This example shows why an equality multiplier can have either sign. Use the same quadratic distance cost and a unit-gradient equality:

$$h(\mathbf{z}) = \frac{z_1 + z_2 - 1}{\sqrt{2}}, \qquad \nabla h = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Stationarity and feasibility give

$$\begin{aligned} (\mathbf{z} - \mathbf{z}_t) + \kappa \nabla h = \mathbf{0}, \qquad h(\mathbf{z}) = 0, \\ \kappa^* = h(\mathbf{z}_t), \\ \mathbf{z}^* = \mathbf{z}_t - \kappa^* \nabla h. \end{aligned}$$

Targets on opposite sides produce opposite multiplier signs:

$\mathbf{z}_t$	κ^*	$\mathbf{z}^*$
$(1.6, 0.6)$	$+0.8485$	$(1, 0)$
$(-0.3, -0.4)$	-1.2021	$(0.55, 0.45)$

The module displays both cases.

(b) The elbow limit (§4.6). The exact IK solutions violate the sixty-degree elbow range. The target and the unconstrained elbow angles are

$$\mathbf{y}^* = (0.5, 0.2)^\top \text{ m}, \qquad \|\mathbf{y}^*\| = 0.5385 \text{ m}, \qquad q_2 = \pm 80.41^\circ.$$

On the positive-elbow branch, the constrained solution is

$$\mathbf{q}^* = (-3.48^\circ, 60.00^\circ)^\top, \qquad \varphi(\mathbf{q}^*) = (0.5648, 0.2259)^\top \text{ m}.$$

Its error, gradient, multiplier and cost are

$$\begin{aligned} \mathbf{r} = (0.0648, 0.0259)^\top \text{ m}, \quad \|\mathbf{r}\| = 0.0698 \text{ m}, \\ \nabla f(\mathbf{q}^*) \approx (0, -0.0119)^\top, \qquad \lambda^* = 0.01192, \\ f^* = 0.002433. \end{aligned}$$

Changing the free first-joint angle by 0.01 rad in either direction increases the cost:

$$f(q_1^* \pm 0.01, 60^\circ) = 0.002450 > f^*.$$

This checks curvature in one direction. It does not establish positive curvature in every relevant direction; the saddle in example (m) can also pass individual coordinate checks. `fig/arm_joint_limit.png` shows this configuration.

(c) The mirror solution. The negative-elbow branch has the same optimal cost:

$$\mathbf{q} = (47.09^\circ, -60^\circ)^\top, \qquad f = 0.002433.$$

The lower limit is active, with

$$\nabla g_2 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \qquad \lambda_2^* = \frac{\partial f}{\partial q_2} = 0.01192.$$

Both are constrained minima, and a local solver's initial configuration can select either.

(d) The price of the limit (17). Relaxing the upper elbow limit by one degree shows how the multiplier predicts a small cost change. Raising it from sixty to sixty-one degrees gives

$$f^* : 0.002433 \longrightarrow 0.002228, \qquad \Delta f^* = -0.000205.$$

Convert the angle change to radians before applying the sensitivity:

$$\Delta f^* \approx -\lambda^* \frac{\pi}{180} \approx -0.01192(0.01745) = -0.000208.$$

The 1.4 percent difference is the higher-order effect of the finite one-degree step.

(e) An inactive limit. Moving the target outward makes the unconstrained solution feasible:

$$\begin{aligned} \mathbf{y}^* &= (0.6, 0.15)^\top \text{ m}, \\ \|\mathbf{y}^*\| &= 0.6185 \text{ m} > R = 0.6083 \text{ m}, \\ q_2 &= \pm 56.49^\circ. \end{aligned}$$

If we incorrectly pin the elbow at the upper boundary, stationarity gives

$$\lambda = -\frac{\partial f}{\partial q_2} = -0.00174 < 0.$$

That candidate fails dual feasibility. At the true solution the limit is inactive, so the multiplier and task-error cost are both zero. The target radius has increased by about eight centimeters relative to (b).

(f) Squared penalty (10) on (b). This run demonstrates decreasing violation at the cost of increasing penalty weight. It uses the two-sided elbow range and starts from

$$\mathbf{q}^{(0)} = (30^\circ, 45^\circ)^\top.$$

Each inner Newton solve is warm-started from the preceding result.

μ	q_1, q_2 (deg)	Violation g_1 (deg)	f	$2\mu g_1$
1	$-3.621,\ 60.338$	$+0.3382$	0.002363	0.01181
10	$-3.497,\ 60.034$	$+0.0341$	0.002426	0.01191
100	$-3.485,\ 60.003$	$+0.0034$	0.002433	0.01192
1000	$-3.484,\ 60.000$	$+0.0003$	0.002433	0.01192

The multiplier estimate uses radians, although the violation column shows degrees:

$$0.338^\circ \approx 0.0059 \text{ rad}, \qquad 2(1)(0.0059) \approx 0.0118.$$

The violation decreases by about ten for each tenfold weight increase, as (21) predicts. The task-error cost rises toward the constrained minimum because the finite-penalty solutions still exceed the joint limit.

(g) Log barrier (11) on (b). This run approaches the active boundary from inside. The two-sided barrier parameter is scaled to the task-error cost.

μ	q_1, q_2 (deg)	Slack $-g_1$ (deg)	f	$-\mu/g_1$
0.01	$9.488,\ 28.833$	31.17	0.009786	0.01838
0.001	$-1.767,\ 55.786$	4.21	0.003356	0.01360
10^{-4}	$-3.292,\ 59.527$	0.47	0.002532	0.01212
10^{-5}	$-3.464,\ 59.952$	0.048	0.002443	0.01194
10^{-6}	$-3.482,\ 59.995$	0.0048	0.002434	0.01192

Every iterate is strictly feasible. Slack is displayed in degrees but converted to radians for the multiplier estimate. Near the solution, slack falls approximately tenfold with a tenfold parameter reduction, consistent with relaxed complementarity:

$$\lambda g = -\mu.$$

(h) Augmented Lagrangian (12)–(13) on (b). This run learns the multiplier while keeping the penalty weight fixed at one.

Outer i	λ_1 before	q_1, q_2 (deg)	g_1 (deg)	λ_1 after	λ_2	Inner steps and outcome
0	0	$-3.621,\ 60.338$	$+0.33819$	0.011805	0	10, stationary
1	0.011805	$-3.485,\ 60.003$	$+0.00321$	0.011917	0	4, stationary
2	0.011917	$-3.484,\ 60.000$	$+0.00003$	0.011918	0	4, stationary
3	0.011918	$-3.484,\ 60.000$	$+0.00000$	0.011918	0	3, stationary
4	0.011918	$-3.484,\ 60.000$	$+0.00000$	0.011918	0	1, stationary

The same outer iterations give the following residuals with the updated multipliers. Feasibility uses the positive violation in radians.

Outer i	Feasibility g_1^+	Stationarity	Complementarity
0	5.9×10^{-3}	4.3×10^{-10}	7.0×10^{-5}
1	5.6×10^{-5}	4.8×10^{-10}	6.7×10^{-7}
2	5.3×10^{-7}	1.4×10^{-11}	6.3×10^{-9}
3	4.9×10^{-9}	1.3×10^{-11}	5.9×10^{-11}
4	6.5×10^{-11}	5.0×10^{-11}	7.7×10^{-13}

The objective-gradient norm remains 0.0119 throughout. The run stops at outer iteration 4 because all three KKT residuals meet their tolerances. The first inner solve equals the first penalty result in (f). Its violation, expressed in radians, becomes the first multiplier update. By the third outer iteration the violation is of order

$$10^{-5} \text{ degrees},$$

although the fixed-weight penalty result missed the limit by about 0.34°. The lower-limit multiplier stays zero because that limit remains inactive. The projection in (13) enforces this behavior. `fig/three_methods.png` compares (f), (g) and (h).

(i) KOMO on the two-link arm (§3.4, §4.9–4.10). This example shows that one trajectory program can have different local solutions. Its fixed start, target and obstacle are

$$\begin{aligned} T &= 20, \qquad k = 2, \qquad \mu = 10, \\ \mathbf{q}_0 &= (30^\circ, 45^\circ)^\top, \\ \mathbf{y}^* &= (0.5, 0.2)^\top \text{ m}, \\ \mathbf{c} &= (0.49, 0.35)^\top \text{ m}, \qquad r_o = 0.05 \text{ m}. \end{aligned}$$

At each knot, clearance is checked at the tip, elbow and midpoint of link 2. These three sampled points approximate the arm geometry. The elbow limits from (b) are absent because they would make this exact goal infeasible.

Guess A interpolates in joint space to the positive-elbow IK solution:

$$\mathbf{q}_T = (-11.52^\circ, 80.41^\circ)^\top.$$

Its initial acceleration cost is 0.001134 and its deepest obstacle penetration is 4.8 cm. Selected outer iterations are

i	Inner steps and outcome	f	$\max_j \lvert h_j \rvert$	$\max_i g_i^+$	Stationarity	Complementarity
0	15, converged	0.000921	9.7×10^{-4}	4.5×10^{-4}	7.4×10^{-12}	4.0×10^{-6}
1	9, converged	0.000945	3.1×10^{-5}	1.1×10^{-5}	2.9×10^{-12}	1.0×10^{-7}
2	8, converged	0.000946	1.4×10^{-6}	1.7×10^{-7}	9.6×10^{-12}	1.6×10^{-9}
5	3, converged	0.000946	5.1×10^{-10}	0	1.3×10^{-10}	9.5×10^{-13}

Each listed iteration has two included obstacle constraints. Its largest multiplier is 0.0090 at iteration 0 and 0.0092 at iterations 1, 2 and 5. The final endpoint reaches the target and has cost

$$\varphi(\mathbf{q}_T) = (0.5, 0.2)^\top \text{ m}, \qquad f = 0.000946.$$

Only the tip constraints at knots 12 and 13 are active, with multipliers 0.0092 and 0.0014. Every other inequality multiplier is zero. The closest sampled-point clearance at the knots is

$$6 \times 10^{-12} \text{ m}.$$

The KKT checks give

$$\lVert \nabla f + \mathbf{J}_h^\top \boldsymbol{\kappa} + \mathbf{J}_g^\top \boldsymbol{\lambda} \rVert = 1.3 \times 10^{-10}, \qquad \max_i \lvert \lambda_i g_i \rvert = 9.5 \times 10^{-13}.$$

The tip passes below the obstacle. Joint steps rise from 0.85° to 4.11° near the middle and return to zero. The normal matrix's condition number is

$$5.1 \times 10^{3}.$$

Despite the knot residuals, the interpolated tip penetrates by 2.07 mm between knots 12 and 13.

Guess B interpolates to the negative-elbow IK solution:

$$\mathbf{q}_T = (55.12^\circ, -80.41^\circ)^\top.$$

It starts without penetration and reaches the outer residual tolerances in five iterations. Its final cost is 0.003746, no obstacle constraint is active, and the closest sampled clearance is 1.06 cm. The tip passes above the obstacle. Joint steps rise from

1.83° to 9.62° and return to zero. The `--k1` velocity-cost variant gives nineteen equal steps of 6.73° and then a zero final step. It starts and stops abruptly, illustrating the role of the acceleration cost.

The two runs satisfy the KKT tests, but solution A costs about one quarter as much. The initial guess selected the local solution. The terms elbow-up and elbow-down refer to the sign of the elbow angle; below and above describe these tip paths. `fig/komo_traj.png` shows both paths and solution A's multipliers by knot. `fig/komo_band.png` shows the 104-by-40 full feature Jacobian with 5.3 percent nonzeros.

(j) The same program handed to Drake's `MathematicalProgram` and SNOPT. The `--drake` comparison shows that matching a problem does not require two algorithms to choose the same local solution. From guess B, SNOPT gives the same cost and a trajectory within 0.003° of our augmented-Lagrangian result. Its equality duals match the negative of our equality multipliers to three digits.

From guess A, SNOPT reaches a different KKT point. Its cost is 0.000957, about one percent higher, and some joint values differ by up to 2.9°. The tip constraints are active at knots 11 and 12 instead of 12 and 13. A least-squares fit of multipliers to that active set gives a stationarity residual of about

$$4 \times 10^{-7}.$$

This is the script's residual at SNOPT's point; its relation to SNOPT's internal tolerance was not checked. Restarting the augmented Lagrangian at that point with those multipliers keeps it there. Both are stationary solutions of the same program.

Drake's raw dual signs depend on the constraint bound. Relative to the convention in (6), its lower-bound inequality dual is positive, its upper-bound dual is negative, and its equality dual is negative:

$$\text{lower bound: } +\lambda, \qquad \text{upper bound: } -\lambda, \qquad \text{equality: } -\kappa.$$

The Panda script checks these signs by sensitivity. Its distance constraint uses squared distance, so the reported multiplier is also converted to the distance form. Those conventions must be accounted for when comparing multiplier values.

(k) Inverse kinematics as a nonlinear program on the Franka Panda. The executed `drake/03-ik-as-nlp/` example uses SNOPT through Drake's `InverseKinematics`. It minimizes distance from the ready posture, imposes the flange-position equality and keeps the model's joint limits:

$$f(\mathbf{q}) = \tfrac{1}{2}\|\mathbf{q} - \mathbf{q}_{\text{ready}}\|^2.$$

The posture $\mathbf{q}_{\text{ready}}$ is this example's fixed reference posture. The frame is the Panda flange, `panda_link8`, and positions are expressed in world axes.

Run 1 reaches

$$\mathbf{y}^* = (0.45, 0.10, 0.45)^\top \text{ m}, \qquad f = 0.209.$$

No joint is at a limit, and every joint-limit multiplier is zero. The position-equality multipliers are

$$\boldsymbol{\kappa} = (-2.0705, -0.6748, 0.7056)^\top.$$

Moving the target by one millimeter in the world x direction increases the cost by 0.00208. The sensitivity prediction is

$$-\kappa_x(0.001) = +0.00207.$$

An equality multiplier can be zero when the cost does not favor crossing its surface, as in the module with the target on the line. It is nonzero for this run.

Run 2 reaches a target behind the robot:

$$\mathbf{y}^* = (-0.20, 0, 0.60)^\top \text{ m}, \qquad f = 1.729,$$

$$\mathbf{q}^* = (0, -1.7628, 0, -2.2173, 0, -0.0046, 0.785)^\top \text{ rad}.$$

Joint 2 is at its lower bound of -1.7628 rad. Its multiplier is about 4.196. The script solves (2) by least squares using the objective gradient, the three-by-seven position Jacobian and the lower-bound gradient:

$$\nabla g = -\mathbf{e}_2.$$

The vector $\mathbf{e}_2$ selects joint 2. The fitted multipliers and residual are

$$\begin{aligned} \lambda &= 4.19613, \\ \boldsymbol{\kappa} &= (8.196, 0, 14.929)^\top, \\ \text{stationarity residual} &= 4 \times 10^{-11}. \end{aligned}$$

Raising the lower limit by 0.001 rad raises the optimal cost by 0.00422. The prediction is

$$\lambda(0.001) \approx 0.00420.$$

A larger perturbation can change the active set. The multiplier gives a local cost derivative rather than a finite-step reachability guarantee; the sensitivity check above tests only the stated 0.001 rad change.

Run 3 uses the run-1 target and an obstacle sphere:

$$\mathbf{c} = (0.45, 0.10, 0.53)^\top \text{ m}, \qquad r_o = 0.07 \text{ m}.$$

The cost rises to 0.386. The wrist frame `panda_link7` is exactly 0.0700 m from the center, with an active distance-form multiplier of 7.30. The other two sampled points are clear at distances 0.080 and 0.077 m. The equality multipliers are

$$\boldsymbol{\kappa} = (2.63, -5.76, -0.02)^\top.$$

In `--meshcat` mode, the displayed active set uses primal gaps. Its thresholds are

$$\text{joint-bound gap} < 10^{-5}\ \text{rad}, \qquad \text{wrist-sphere gap} < 10^{-6}\ \text{m}.$$

SNOPT can leave raw duals between the following values on inactive bounds:

$$10^{-5} \text{ to } 10^{-4}.$$

The display therefore does not infer activity from a nonzero raw dual alone.

(l) Deck 04's clamped point. Clipping the elbow after each Gauss-Newton step satisfies the range but does not enforce constrained stationarity. That run stalls at

$$\begin{aligned} \mathbf{q} &= (-16.40^\circ, 60^\circ)^\top, \\ f &= 0.0107 = 4.4 f^*, \\ \nabla f &= (-0.0732, -0.0439)^\top. \end{aligned}$$

The nonzero first component allows improvement along the free joint. KKT stationarity therefore rejects this point even though the elbow lies on the boundary.

(m) KKT points that are not minima, and a minimum where KKT fails. These cases show why both the regularity assumption and a separate minimum test matter. The straight arm pointing past the target is

$$\mathbf{q} = (21.80^\circ, 0^\circ)^\top, \qquad \nabla f = \mathbf{0}, \qquad f = 0.01304 = 5.36 f^*.$$

No limit is active, and every KKT condition holds. Nevertheless this point is a saddle. The barrier failure in §7 (6) also stops there. On the upper boundary, another KKT point is

$$\mathbf{q} = (176.52^\circ, 60^\circ)^\top, \qquad \frac{\partial f}{\partial q_1} = 0, \qquad \lambda = -\frac{\partial f}{\partial q_2} = +0.196.$$

It is a maximum along the boundary rather than a minimum.

For the stretched arm at zero joint angles, the exact goal equality has Jacobian

$$\varphi(\mathbf{q}) = (0.7, 0)^\top, \qquad \mathbf{J}(0,0) = \begin{bmatrix} 0 & 0 \\ 0.7 & 0.3 \end{bmatrix}.$$

The stretched configuration is locally the only feasible point for that target, so it is a constrained minimum for any cost defined there. The Jacobian has rank one: one equality gradient is zero, and the two are linearly dependent. No equality multipliers

can balance a cost gradient outside their span. The constraint qualification fails, so the KKT necessity result does not apply.

---

6. Code

Run the numerical examples from the repository root with the course Python environment:

```
.venv/bin/python \
  lectures/04-D1-constraints-and-komo/code/constrained_ik.py
.venv/bin/python \
  lectures/04-D1-constraints-and-komo/code/komo_two_link.py
.venv/bin/python \
  lectures/04-D1-constraints-and-komo/code/komo_two_link.py --drake
.venv/bin/python \
  lectures/04-D1-constraints-and-komo/code/komo_two_link.py --k1
.venv/bin/python \
  lectures/04-D1-constraints-and-komo/code/komo_two_link.py --sweep
.venv/bin/python \
  drake/03-ik-as-nlp/ik_nlp_panda.py
```

`constrained_ik.py` implements examples (a)–(h), (l) and (m). Its shared `newton_armijo` function solves each unconstrained inner problem with Armijo backtracking and the Gauss-Newton Hessian model plus that method's constraint curvature. It returns `(z, iterations, outcome)`. Its limits are

$$\begin{aligned}\text{gradient tolerance} &= 10^{-9},\\ \text{line-search floor} &= 10^{-12},\\ \text{iteration cap} &= 200.\end{aligned}$$

A rejected floor trial leaves the iterate unchanged and returns `stalled`. The augmented-Lagrangian outer loop checks the residuals of §4.11. The script takes about one second.

`komo_two_link.py` implements (9). Its `acc_residuals`, `eq_constraints` and `ineq_constraints` functions return values and Jacobians. The `al_residual` function constructs the shifted residuals from §4.9. The inner `gauss_newton` function uses damping and Armijo backtracking from a unit step. It returns `converged` on its accepted-step test, `stalled` on a rejected floor trial, or `limit` at its cap:

$$\begin{aligned}\text{step tolerance} &= 10^{-10},\\ \text{line-search floor} &= 10^{-8},\\ \text{iteration cap} &= 100.\end{aligned}$$

The `solve_komo` outer loop applies (13), checks all three KKT residuals and propagates an inner stall. The printed output includes joint steps, between-knot clearance, conditioning and both Jacobian shapes. The flags compare SNOPT, a velocity cost and three knot counts respectively. A normal run takes about two seconds, or about ten seconds with `--drake`.

`check_solvers.py` checks rejected trials, propagated stalls, the numerical Jacobians and the reported results. One rejection test uses

$$f(z) = \tfrac{1}{2}\,10^{14} z^2, \qquad H_{\text{model}} = 1, \qquad z^{(0)} = 1.$$

`newton_armijo` must return the original iterate with `stalled`. A stiff-residual test checks the same behavior in `gauss_newton` and the outer `solve_komo` loop. Central differences check the three Jacobians of (9) at solution A with errors below

$$2 \times 10^{-10}.$$

The shape checks expect 104 by 40 for the full feature Jacobian and 46 by 40 for the included inner residual Jacobian. The repository check runs this script. `make_figures.py` generates the figures from the same functions.

The Panda IK script prints example (k) and its least-squares multiplier check. Add `--meshcat` to adjust the target and re-solve while displaying the active set. The Drake guide and this deck's demo guide give setup and complete controls.

The KKT explorer lets you drag the target for the disc, halfspace and equality examples. Predict which constraint is active and which sign its multiplier should have, then compare the gradient arrows and KKT checks. The figures in example (a) provide a static version.

The inherited optimization explorer has Constrained and Trajectory tabs. Its labels use x for the decision variable, λ for damping and ρ for the penalty weight. Translate these to our $\mathbf{z}$, ρ and μ, respectively. Its multiplier display uses its own contextual labels; read them within the selected constrained problem.

For the Constrained tab, choose **Circle Constraint** and **Augmented Lagrangian**. At fixed unit penalty weight, the multiplier approaches one. Its first displayed values are

$$0, \quad 0.716, \quad 0.910, \quad 0.970, \quad 0.990.$$

Compare growth factors one and five. The outer step counts depend on the tolerance:

Tolerance	Growth one	Growth five
10^{-6}	13	5
10^{-2} (default)	5	4

The initial-weight slider stops at ten, so it does not reproduce example (f)'s full penalty sweep. The companion module notes contain the exact starting states and controls.

For the Trajectory tab, choose **Between-knot collision**. The four knots have first-joint angles $-60°$, $-20°$, $20°$ and $60°$, with the second joint fixed at zero. All four knot configurations are clear. The preset then compares those checks with samples of the same joint-linear interpolation:

Check	Minimum clearance in workspace units	Obstacle cost
Original four knots	$+0.0428$	0
61 interpolated samples	-0.0344	—
Seven knots after **Add midpoint knots**	-0.0121	0.0147
121 interpolated samples	-0.0400	—

Adding midpoint knots keeps the same path and reveals a collision at the new knots. The blue dashed curve is forward kinematics of the interpolated joint path, and the red dashed arm shows the worst sampled collision. This module checks full link segments; example (i)'s Python program checks three points. Negative clearance establishes collision in the model. Positive sampled clearance does not certify continuous clearance. The named preset resets its geometry, weights and constraints. Trajectory augmented Lagrangian is disabled pending correction; it remains available on the Constrained tab. No optimizer step is needed for this collision demonstration.

KOMO in Drake, line by line (executed). The function `drake_cross_check` in `komo_two_link.py` constructs the program of §3.4 with Drake's `MathematicalProgram`. The acceleration residual is linear, so `Add2NormSquaredCost` adds the smoothness cost. One `AddConstraint` with equal zero lower and upper bounds imposes the final goal and rest equalities. Another uses a negative-infinite lower bound and zero upper bound for all obstacle inequalities. `SnoptSolver().Solve` runs SNOPT from the same initial guess as our outer loop, and `GetDualSolution` returns Drake's raw duals. SNOPT is a sequential quadratic programming solver.

From guess A, the executed comparison gives cost 0.000957 against 0.000946, joint values differing by up to 2.88°, and active obstacle rows 30 and 33 against 33 and 36. It has no violation and fitted stationarity residual

$$3.6 \times 10^{-7}.$$

Restarting our augmented Lagrangian with those fitted multipliers moves by only 0.0001° in five outer iterations. From guess B, the two paths agree within 0.003°. The execution record is `planning/consolidation-2026-09-29/review/komo_two_link_output_2026-09-29-stageD.txt`. The comparison matches model, features, weights, time convention and initial guess; its different stationary paths illustrate local-solver behavior.

KOMO in RAI (documentation only). The following mapping shows how the same feature types enter the `robotic` package. RAI has no macOS wheel in the course environment, so these calls have not been executed here. They were checked against the package documentation and `komo_1_intro` tutorial on 2026-09-06 and 2026-09-28. In particular, `FS.jointState` with `order=1`, `FS.negDistance` and the collision setting need a RAI-specific execution check before this mapping is relied upon.

Here `C` is a configured RAI scene containing the `l_gripper`, `target` and `obstacle` frames, and `ry` denotes the imported `robotic` package.

```
# The problem has twenty slices and order-two features.
komo = ry.KOMO(
    C, phases=1, slicesPerPhase=20,
    kOrder=2, enableCollisions=True,
)
# The cost penalizes acceleration differences.
komo.addControlObjective([], order=2, scale=1.0)
# The gripper reaches the target at the last knot.
komo.addObjective(
    [1.], ry.FS.positionDiff,
    ['l_gripper', 'target'], ry.OT.eq, [1e1],
)
# The final joint difference is zero, representing rest.
komo.addObjective(
    [1.], ry.FS.jointState, [], ry.OT.eq, [1e1], order=1,
)
# The gripper maintains obstacle clearance at each knot.
komo.addObjective(
    [], ry.FS.negDistance,
    ['l_gripper', 'obstacle'], ry.OT.ineq, [1e1],
)
# The documented solver uses an augmented Lagrangian.
ret = ry.NLP_Solver(komo.nlp(), verbose=0).solve()
print(ret.feasible, ret.eq, ret.ineq, ret.sos)
q_path = komo.getPath()
```

`ry.OT.sos` denotes a sum-of-squares cost, `ry.OT.eq` an equality and `ry.OT.ineq` an inequality with the nonpositive-feasibility convention. Times are in phase units: `[1.]` selects the final knot and `[]` selects every knot. The `order` argument selects the feature's temporal order.

A scale multiplies the feature. For a sum-of-squares feature, its effective weight is the square of that scale, as with the square-root weight matrix in Gauss-Newton. For a positive scalar scale s on an equality or inequality,

$$\tilde{h} = sh \quad \text{or} \quad \tilde{g} = sg, \qquad \tilde{\kappa} = \frac{\kappa}{s} \quad \text{or} \quad \tilde{\lambda} = \frac{\lambda}{s}, \qquad \mu_{\text{effective}} = \mu s^2.$$

The feasible set is unchanged, but reported multipliers and the effective penalty weight change. At a loose numerical stopping tolerance, this can change the achieved residual. `addControlObjective` internally scales by the square root of the time step.

The documentation-based mapping does not treat `enableCollisions=True` as a collision objective by itself. The shown `negDistance` feature protects only the gripper; example (i) instead protects three points. An `OT.sos` target is a soft cost, so missing that target alone is not an infeasibility condition. Other hard constraints can still make the program infeasible. An `OT.eq` target is a hard requirement. If it is unreachable, the program has no feasible solution. The solver's `ret.feasible` remains a report about that run, as explained in §3.2.

---

7. Common failure modes

1. An obstacle constraint needs the correct sign. It should be positive in the forbidden region and nonpositive outside. Reversing the distance subtraction makes the inside of the obstacle feasible.

2. A negative inequality multiplier fails dual feasibility. Reconsider the proposed active set or minimum. Equality multipliers can be negative, so their sign should not be projected or reversed.

3. A constraint inactive at the start may become active later. Guess A in example (i) penetrates by 4.8 cm at knot 10 before its first outer solve. Reevaluate the included rows during the iterations and identify activity at the solution.

4. A small step in a finite-penalty solve does not establish feasibility. At weight ten, example (f) still exceeds the elbow limit by 0.034°. Check the violation separately.

5. A log barrier requires a strictly feasible start and cannot encode an equality. The inherited explorer disables Log Barrier for its equality problem for this reason.

6. A barrier weight too large for the cost can lead to an unwanted stationary branch. On the elbow example, a starting weight of 0.1 dominates a cost near 0.002 and sends the first solve to the midpoint of the elbow range. The symmetric straight-arm configuration has zero elbow derivative. Newton then stays at that saddle for smaller parameters and reports

$$q_2 = 0.000^\circ, \qquad f = 0.0130,$$

about five times the constrained minimum, with no error message. This midpoint is the barrier's analytic center. The schedule in example (g) starts at 0.01 and stays in the positive-elbow solution basin. Strict feasibility alone does not prevent a saddle.

7. Two local solvers can return different stationary paths for the same obstacle problem. Example (j) has costs one percent apart and different active contact knots. Report the initial guess and measured residuals with the answer.

8. Three sampled points do not cover the whole arm, and knot checks do not cover the continuous path. A thin obstacle can lie between sampled points. The interpolation penetration in §3.4 demonstrates the separate temporal issue. RAI collision features and Drake's `AddMinimumDistanceLowerBoundConstraint` use full geometry, but a trajectory still needs checks between knots. Denser knots reduce penetration in this example; they do not by themselves prove continuous clearance.

9. The symbol k denotes feature order here. Use i for iterations, so a solver iteration cannot be confused with the feature's time dependence.

10. The horizon T counts free knots here. It is not a duration in seconds. State the time step and residual scaling before interpreting a cost or comparing multipliers.

11. A soft target and a hard target define different problems. `OT.sos` permits target error at a cost; `OT.eq` requires the target equality. A soft target does not remove infeasibility caused by other hard constraints.

12. The unconstrained objective-gradient test does not apply to the outer constrained problem. Constraint gradients can balance a nonzero objective gradient at the optimum. Use feasibility, Lagrangian stationarity and complementarity as in §4.11.

---

8. Where this sits in the stack

Session 4.1 supplies the optimality conditions for goals with joint limits and obstacle clearance. Session 4.2 applies them to a complete reference trajectory. The KOMO program combines the smoothness cost with exact goals and sampled clearance constraints.

Session 4.3 uses the same constraint and multiplier logic in a differential-IK quadratic program with velocity limits. That controller follows a reference motion. Session 4.4 adds sampling-based planning as another way to construct the reference. Later, session

5.1 studies learned proposals for configurations or paths, and session 7.1 uses behavior cloning to learn from expert motions.

Learning from a constrained expert does not automatically enforce its constraints in new situations. A policy's predicted motion still needs appropriate feasibility and collision checks. The classical multipliers also quantify which active requirement limits the objective and its local cost sensitivity, as the elbow and Panda examples show.

9. Further reading (optional, never load-bearing)

- Jorge Nocedal and Stephen Wright's *Numerical Optimization* gives the KKT conditions in chapter 12, penalty and augmented-Lagrangian methods in chapter 17, and interior-point methods in chapter 19. It supplies the regularity and local-convergence assumptions behind the derivations here.
- Stephen Boyd and Lieven Vandenberghe's *Convex Optimization* explains duality and multiplier sensitivity in chapter 5, and the barrier method and central path in chapter 11. Its global barrier guarantees require the convex setting.
- Marc Toussaint's *A tutorial on Newton methods for constrained trajectory optimization and relations to SLAM, Gaussian process smoothing, optimal control, and probabilistic inference* (2017) develops the k-order formulation and banded Gauss-Newton step.
- Marc Toussaint's *A novel augmented Lagrangian approach for inequalities and convergent any-time non-central updates* (arXiv:1412.4329, 2014) describes the inequality variant and multiplier update used in (12)–(13).
- Russ Tedrake's *Robotic Manipulation* connects differential IK to a quadratic program in chapter 3 and develops further trajectory-optimization formulations.
- The frozen Spring 2026 sources in `source-2026/opt-lectures/05-07*.md` give a fuller treatment. Their central-path and saddle-point pictures are reused in the backup slides.

Open questions and weak points

- The derivations establish local conditions. KKT residuals alone cannot distinguish a minimum from a saddle or prove that a solver has found the best trajectory.
- The Python trajectory checks three arm points at discrete knots. Full-shape and continuous-path clearance require additional collision checks.
- The RAI mapping is documentation-based and has not been executed in this environment. Its feature names, collision behavior, weight scaling and solver defaults need a separate RAI-specific check. Executed Drake results do not verify these calls.

The next sessions use this optimization machinery to follow reference motions and compare trajectory optimization with sampling-based planning.